

```

class Student:
    def abc(self):
        return 'Student class'

s1 = Student()
s1.abc()
'Student class'

class Student:
    def __init__(self, e, m, s):
        self.e = e
        self.m = m
        self.s = s

    def get_m(self): #getters, accessors
        return f'{self.m}'
    def set_m(self, val): # setters, mutators
        self.m = val
    def perc(self):
        return f'{(self.e+self.m+self.s)/3} %'

s1 = Student(76, 98, 67)
s1.get_m()
'98'
s1.set_m(32)
s1.get_m()
'32'
s1 = Student(76, 87, 98)
s1.get_m()
'87'
s1.m
87
s1.perc()
'87.0 %'

class Student:
    count = 0
    def __init__(self, e, m, s):
        self.e = e

```

```

        self.m = m
        self.s = s
        Student.count += 1

    def perc(self):
        return f'{(self.e+self.m+self.s)/3} %'

    @classmethod
    def get_count(cls):
        return f'total = {cls.count}'

```

Student.count

0

s1 = Student(76, 78, 87)

s1.get_count()

'total = 1'

s2 = Student(6, 8, 7)

s3 = Student(73, 77, 97)

s1.get_count()

'total = 3'

Student.get_count()

'total = 3'

```

class Student:
    count = 0
    total_fee = 0
    def __init__(self, name, e, m, fee):
        self.e = e
        self.m = m
        self.name = name
        self.fee = fee
        Student.count += 1
        Student.total_fee += fee
    @classmethod
    def get_count(cls):
        return f'total = {cls.count}'
    @classmethod
    def get_totalfee(cls):
        return f'Total collection: {cls.total_fee}'

```

```
s1 = Student('anurag', 87, 89, 23400)
s2 = Student('amit', 67, 56, 12000)
s3 = Student('rahul', 45, 76, 54000)
```

```
s1.get_totalfee()
```

```
'Total collection: 89400'
```

```
class Parent:
    def a(self):
        print('Base Class')
```

```
class Child:
    def a(self):
        print('Child Class')
```

```
c1 = Child()
```

```
c1.a()
```

```
Child Class
```

```
class Parent:
    def a(self):
        print('Base Class')
```

```
class Child:
    def a(self):
        super().a()
        print('Child Class')
```

```
c1 = Parent()
```

```
dir(c1)
```

```
['_class__',
 '__delattr__',
 '__dict__',
 '__dir__',
 '__doc__',
 '__eq__',
 '__format__',
 '__ge__',
 '__getattr__',
 '__getstate__',
 '__gt__',
 '__hash__',
 '__init__',
 '__init_subclass__',
 '__le__',
 '__lt__',
 '__module__',
```

```
'__ne__',
'__new__',
'__reduce__',
'__reduce_ex__',
'__repr__',
'__setattr__',
'__sizeof__',
'__str__',
'__subclasshook__',
'__weakref__',
'a']
```

```
class Parent:
    def a(self):
        print('Base Class')
```

```
class Child(Parent):
    def a(self):
        super().a()
        print('Child Class')
```

```
c1 = Child()
```

```
c1.a()
```

```
Base Class
Child Class
```

```
class A:
    def __init__(self, name):
        self.name = name
        # self.age = age
        print(self.name)
```

```
class B(A):
    def __init__(self, name, age):
        super().__init__(name)
        self.age = age
        print('child')
```

```
b1 = B('nashit', 23)
```

```
nashit
child
```

```
import math
```

```
import sys
```

```
import os
```

```
import numpy
```

```

import numpy as np
l = [7,8,3]
a = np.array(0)
type(a)
numpy.ndarray
a1 = np.array([6, 7, 4])
type(a1)
numpy.ndarray
a1.ndim
1
a1.shape
(3,)
l = [[8, 7], [5, 4]]
a2 = np.array([[8, 7, 9], [5, 4, 3]])
a2.ndim
2
a2.shape
(2, 3)
a3 = np.array([[8, 7, 9], [5, 4, 3]])
a4 = np.array([
    [1, 2],
    [11, 22]]
])
a4.ndim
3
a4.shape
(1, 2, 2)
a4 = np.array([8,
               7,
               9])

```

```
a4.shape
(3,)
a5 = np.array([2, 4, 7, 8], ndmin = 6)
a5
array([[[[[[2, 4, 7, 8]]]]]])
a1 = np.array([4, 8, 5, 2])
print(a1[0])
4
print(a1[-1])
2
print(a1[0:2])
[4 8]
print(a1[:,2])
[4 5]
a1[0] = 88
a1
array([88, 8, 5, 2])
a1 = np.array([1, 2, 3, 4])
a2 = np.array([12, 11, 14, 15])
a1+a2
array([13, 13, 17, 19])
a3 = np.array([[2,3,4,5],[12,13,14,15]])
a3
array([[ 2,  3,  4,  5],
       [12, 13, 14, 15]])
a3[0]
array([2, 3, 4, 5])
a3[0,0]
np.int64(2)
```

```

print(a3[0][0])
2
a1
array([1, 2, 3, 4])
a1.dtype
dtype('int64')
a1 = np.array([9.1, 2.0, 3.0])
a1.dtype
dtype('float64')
a1 = np.array(['a', 'b'])
a1.dtype
dtype('<U1')
a1 = np.array([1, 2, 6, 4, 8, 9, 2, 5, 4, 6])
a1.shape
(10,)
a1.ndim
1
a1.reshape(5,2)
array([[1, 2],
       [6, 4],
       [8, 9],
       [2, 5],
       [4, 6]])
a1.reshape(3,3)

```

```

-----
-----
ValueError                                Traceback (most recent call
last)
Cell In[59], line 1
----> 1 a1.reshape(3,3)

ValueError: cannot reshape array of size 10 into shape (3,3)

```